

Informe de práctica: Fragmentación de base de datos distribuida en PostgreSQL

Equipo de trabajo

2026

1. Introducción

La práctica desarrollada consistió en implementar un escenario de base de datos distribuida usando PostgreSQL dentro de contenedores Docker. El objetivo principal fue aplicar fragmentación horizontal, vertical y mixta sobre una base de datos de cafetería universitaria. Para ello se trabajó con tres nodos: **pg-campus**, **pg-babahoyo** y **pg-ventanas**. Cada nodo representa una sede donde se aloja una parte de los datos.

La práctica permitió comprobar tres condiciones importantes de una fragmentación correcta: completitud, reconstrucción y disjunción. La completitud asegura que ningún dato se pierda; la reconstrucción permite volver a consultar la información completa desde una vista global; y la disjunción evita que una misma fila se repita innecesariamente en más de un nodo.

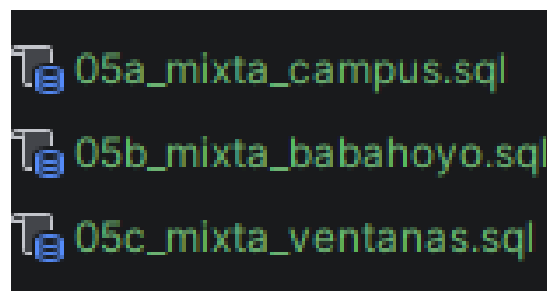
2. Estructura inicial del proyecto

La estructura del proyecto se organizó con una carpeta principal, un archivo `docker-compose.yml` y una carpeta `sql` con los scripts necesarios para crear el esquema, insertar datos, fragmentar y verificar la base de datos. Durante la preparación se identificó que el archivo `05_fragmentacion_mixta.sql` contenía instrucciones que debían ejecutarse en nodos diferentes. Por ese motivo se decidió conservar el archivo general y, además, crear tres scripts separados para ejecutar la fragmentación mixta de forma segura en cada contenedor, como se observa en las Evidencias [1](#) y [2](#).



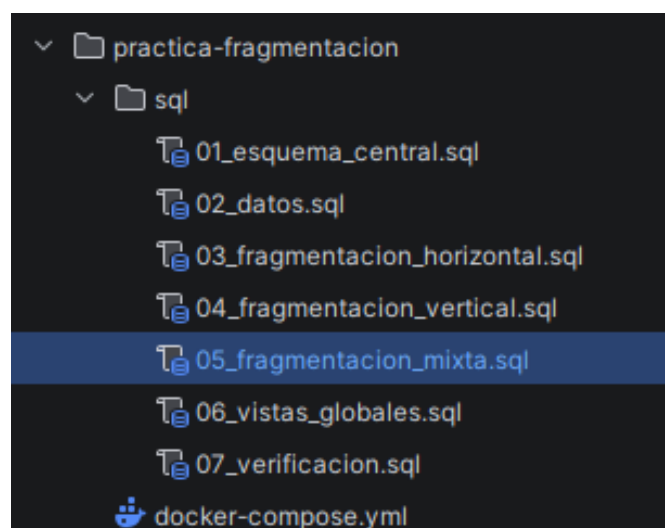
```
1  -- Para pg-campus
2  DROP TABLE IF EXISTS clientes_quevedo_publicos;
3
4  CREATE TABLE clientes_quevedo_publicos (
5      cliente_id INTEGER PRIMARY KEY,
6      nombre VARCHAR(80) NOT NULL,
7      ciudad VARCHAR(40) NOT NULL,
8      CHECK (ciudad = 'Quevedo')
9  );
10
11 INSERT INTO clientes_quevedo_publicos (cliente_id, nombre, ciudad) VALUES
12 ( cliente_id 1, nombre 'Maria Alvarado', ciudad 'Quevedo'),
13 ( cliente_id 3, nombre 'Ana Vera', ciudad 'Quevedo');
14
15 SELECT * FROM clientes_quevedo_publicos;
16
17 -- para pg-babahoyo
18 DROP TABLE IF EXISTS clientes_quevedo_contacto;
```

Evidencia 1: Archivo de fragmentación mixta con bloques dirigidos a nodos específicos.



Evidencia 2: Archivos separados para ejecutar la fragmentación mixta en cada contenedor.

En la Evidencia 3 se muestra la carpeta del proyecto con los scripts SQL iniciales. Después de separar los archivos correspondientes, la práctica quedó lista para levantar los contenedores y ejecutar cada script de acuerdo con el nodo correspondiente.



Evidencia 3: Estructura inicial del proyecto con la carpeta SQL y el archivo Docker Compose.

3. Levantamiento de contenedores

Para iniciar el entorno distribuido se utilizó Docker Compose. El comando principal usado fue:

```
docker compose up -d
```

La descarga y creación de los contenedores se evidencia en las Evidencias 4 y 5. Posteriormente, se verificó desde Docker Desktop y desde terminal que los tres nodos estaban levantados correctamente, como se observa en las Evidencias 6 y 7.

```
Terminal Local x Local (2) x + v
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS D:\Programas\Practica_fragmentacion_07-07\practica-fragmentacion> docker compose up -d
[+] Running 0/14
- pg-campus Pulling
- pg-babahoyo [#####] 13.15MB / 116MB Pulling
- 689a434e3a86 Downloading [=====] 116B/116B
- 0749aef11dca Downloading [=====] 970B/970B
- c86ab38aa6db Downloading [=====] 128B/128B
- 55afa1ecc21d Downloading [=====] 3.846MB/3.846MB
- cc7fa208b8a7 Downloading [=====] 184B/184B
- ef6ab46d98b2 Downloading [=====] 173B/173B
- 2d3064b8615b Downloading [=====] 900.3kB/900.3kB
- 713473b5b92b Downloading [=====] 167B/167B
- 7af924000309 Downloading [=====] 6.109kB/6.109kB
- 4096c457dc0d Downloading [=====] 9.62kB/9.62kB
- 2f537278bc74 Downloading [====] 8.389MB/111.3MB
- pg-ventanas Pulling
```

Evidencia 4: Ejecución de `docker compose up -d` para iniciar los contenedores.

```
PS D:\Programas\Practica_fragmentacion_07-07\practica-fragmentacion> docker compose up -d
[+] Running 14/14
✓ pg-campus Pulled
✓ pg-babahoyo Pulled
✓ 689a434e3a86 Pull complete
✓ 0749aef11dca Pull complete
✓ c86ab38aa6db Pull complete
✓ 55afa1ecc21d Pull complete
✓ cc7fa208b8a7 Pull complete
✓ ef6ab46d98b2 Pull complete
✓ 2d3064b8615b Pull complete
✓ 713473b5b92b Pull complete
✓ 7af924000309 Pull complete
✓ 4096c457dc0d Pull complete
✓ 2f537278bc74 Pull complete
✓ pg-ventanas Pulled
[+] Running 7/7
✓ Network practica-fragmentacion_default Created
✓ Volume practica-fragmentacion_pg-ventanas-data Created
✓ Volume practica-fragmentacion_pg-campus-data Created
✓ Volume practica-fragmentacion_pg-babahoyo-data Created
✓ Container pg-ventanas Started
✓ Container pg-babahoyo Started
✓ Container pg-campus Started
```

Evidencia 5: Proceso de descarga, creación de red, volúmenes y contenedores.

☐	☒	practica-fragmentacion	-	-	-	0.09%	11 seconds ago			
☐	☒	pg-campus	e6e0e282892c	postgres:16-alpine	5433:5432	0.03%	11 seconds ago			
☐	☒	pg-ventanas	b6929a7e33d9	postgres:16-alpine	5435:5432	0.03%	11 seconds ago			
☐	☒	pg-babahoyo	cd81553931a9	postgres:16-alpine	5434:5432	0.03%	11 seconds ago			

Evidencia 6: Contenedores visibles en Docker Desktop.

PS D:\Programas\Practica_fragmentacion_07-07\practica-fragmentacion> docker compose ps						
NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
pg-babahoyo	postgres:16-alpine	"docker-entrypoint.s..."	pg-babahoyo	4 minutes ago	Up 4 minutes	0.0.0.0:5434->5432/tcp, [::]:5434->5432/tcp
pg-campus	postgres:16-alpine	"docker-entrypoint.s..."	pg-campus	4 minutes ago	Up 4 minutes	0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
pg-ventanas	postgres:16-alpine	"docker-entrypoint.s..."	pg-ventanas	4 minutes ago	Up 4 minutes	0.0.0.0:5435->5432/tcp, [::]:5435->5432/tcp

Evidencia 7: Verificación de contenedores mediante docker compose ps.

4. Ejecución de scripts SQL

La ejecución de los scripts se realizó desde PowerShell usando `Get-Content`, ya que la redirección con el operador `<` no funciona de la misma manera en PowerShell. El esquema centralizado y la carga de datos se aplicaron sobre el nodo `pg-campus`. La ejecución inicial se observa en las Evidencias 8 y 9.

```
Get-Content .\sql\01_esquema_central.sql | docker exec -i pg-campus psql -U
admin -d cafeteria
Get-Content .\sql\02_datos.sql | docker exec -i pg-campus psql -U admin -d
cafeteria
```

```
PS D:\Programas\Practica_fragmentacion_07-07\practica-fragmentacion> Get-Content .\sql\01_esquema_central.sql | docker exec -i pg-campus psql -U admin -d cafeteria
CREATE TABLE
CREATE TABLE
CREATE TABLE
```

Evidencia 8: Ejecución del script de esquema centralizado.

```
PS D:\Programas\Practica_fragmentacion_07-07\practica-fragmentacion> Get-Content .\sql\02_datos.sql | docker exec -i pg-campus psql -U admin -d cafeteria
INSERT 0 6
INSERT 0 5
INSERT 0 8
```

Evidencia 9: Ejecución del script de carga de datos iniciales.

4.1. Fragmentación horizontal

La fragmentación horizontal se aplicó sobre la tabla `pedidos`, usando la columna `sede`. Cada nodo recibió únicamente los pedidos de su sede correspondiente. En las Evidencias 10, 11 y 12 se muestra la ejecución de los scripts para Campus, Babahoyo y Ventanas.

```
PS D:\Programas\Practica_fragmentacion_07-07\practica-fragmentacion> Get-Content .\sql\03_fragmentacion_horizontal_campus.sql | docker exec -i pg-campus psql -U admin -d cafeteria
DROP TABLE
CREATE TABLE
INSERT 0 3
pedido_id | cliente_id | producto_id | fecha      | monto | sede
-----+-----+-----+-----+-----+-----
1 | 1 | 1 | 2026-03-01 | 0.75 | Campus
3 | 3 | 2 | 2026-03-02 | 1.00 | Campus
7 | 1 | 3 | 2026-03-04 | 2.50 | Campus
(3 rows)
```

Evidencia 10: Fragmentación horizontal ejecutada en el nodo Campus.

```
PS D:\Programas\Practica_fragmentacion_07-07\practica-fragmentacion> Get-Content .\sql\03_fragmentacion_horizontal_babahoyo.sql | docker exec -i pg-babahoyo psql -U admin -d cafeteria
NOTICE: table "pedidos" does not exist, skipping
DROP TABLE
CREATE TABLE
INSERT 0 3
pedido_id | cliente_id | producto_id | fecha      | monto | sede
-----+-----+-----+-----+-----+-----
2 | 2 | 3 | 2026-03-01 | 2.50 | Babahoyo
6 | 6 | 1 | 2026-03-03 | 0.75 | Babahoyo
8 | 2 | 2 | 2026-03-04 | 1.00 | Babahoyo
(3 rows)
```

Evidencia 11: Fragmentación horizontal ejecutada en el nodo Babahoyo.

```
PS D:\Programas\Practica_fragmentacion_07-07\practica-fragmentacion> Get-Content .\sql\03_fragmentacion_horizontal_ventanas.sql | docker exec -i pg-ventanas psql -U admin -d cafeteria
NOTICE: table "pedidos" does not exist, skipping
DROP TABLE
CREATE TABLE
INSERT 0 2
pedido_id | cliente_id | producto_id | fecha      | monto | sede
-----+-----+-----+-----+-----+-----
4 | 4 | 4 | 2026-03-02 | 1.25 | Ventanas
5 | 5 | 5 | 2026-03-03 | 1.00 | Ventanas
(2 rows)
```

Evidencia 12: Fragmentación horizontal ejecutada en el nodo Ventanas.

4.2. Fragmentación vertical

La fragmentación vertical se aplicó sobre la tabla `clientes`. Los datos públicos se almacenaron en `pg-campus` y los datos de contacto se almacenaron en `pg-babahoyo`. La Evidencia 13 muestra la ejecución de esta parte de la práctica.

```
PS D:\Programas\Practica_fragmentacion_07-07\practica-fragmentacion> Get-Content .\sql\04_fragmentacion_vertical_campus.sql | docker exec -i pg-campus psql -U admin -d cafeteria
CREATE TABLE
INSERT 0 6

PS D:\Programas\Practica_fragmentacion_07-07\practica-fragmentacion> Get-Content .\sql\04_fragmentacion_vertical_babahoyo.sql | docker exec -i pg-babahoyo psql -U admin -d cafeteria
CREATE TABLE
INSERT 0 6
```

Evidencia 13: Ejecución de los scripts de fragmentación vertical.

4.3. Fragmentación mixta

La fragmentación mixta combinó fragmentación horizontal y vertical. Primero se separaron los clientes por ciudad y después se separaron sus columnas según el tipo de dato. Debido a que cada fragmento debía ir a un nodo distinto, se usaron tres archivos: `05a_mixta_campus.sql`, `05b_mixta_babahoyo.sql` y `05c_mixta_ventanas.sql`. Los resultados se muestran en las Evidencias 14, 15 y 16.

```
PS D:\Programas\Practica_fragmentacion_07-07\practica-fragmentacion> Get-Content .\sql\05a_mixta_campus.sql | docker exec -i pg-campus psql -U admin -d cafeteria
DROP TABLE
NOTICE: table "clientes_quevedo_publicos" does not exist, skipping
CREATE TABLE
INSERT 0 2
 cliente_id | nombre | ciudad
-----+-----+-----
          1 | Maria Alvarado | Quevedo
          3 | Ana Vera | Quevedo
(2 rows)
```

Evidencia 14: Fragmentación mixta ejecutada en el nodo Campus.

```
PS D:\Programas\Practica_fragmentacion_07-07\practica-fragmentacion> Get-Content .\sql\05b_mixta_babahoyo.sql | docker exec -i pg-babahoyo psql -U admin -d cafeteria
NOTICE: table "clientes_quevedo_contacto" does not exist, skipping
DROP TABLE
NOTICE: table "clientes_otros_contacto" does not exist, skipping
CREATE TABLE
CREATE TABLE
INSERT 0 2
INSERT 0 4
 cliente_id | email | telefono
-----+-----+-----
          1 | maria@uteq.edu.ec | 0991111111
          3 | ana@uteq.edu.ec | 0993333333
(2 rows)

 cliente_id | email | telefono
-----+-----+-----
          2 | luis@uteq.edu.ec | 0992222222
          4 | jose@uteq.edu.ec | 0994444444
          5 | carla@uteq.edu.ec | 0995555555
          6 | pedro@uteq.edu.ec | 0996666666
(4 rows)
```

Evidencia 15: Fragmentación mixta ejecutada en el nodo Babahoyo.

```
PS D:\Programas\Practica_fragmentacion_07-07\practica-fragmentacion> Get-Content .\sql\05c_mixta_ventanas.sql | docker exec -i pg-ventanas psql -U admin -d cafeteria
NOTICE: table "clientes_otros_publicos" does not exist, skipping
DROP TABLE
CREATE TABLE
INSERT 0 4
 cliente_id | nombre | ciudad
-----+-----+-----
          2 | Luis Cedeno | Babahoyo
          4 | Jose Mendoza | Ventanas
          5 | Carla Zambrano | Ventanas
          6 | Pedro Suarez | Babahoyo
(4 rows)
```

Evidencia 16: Fragmentación mixta ejecutada en el nodo Ventanas.

5. Vistas globales y reconstrucción

Las vistas globales se construyeron desde pg-campus mediante `postgres_fdw`. La vista `pedidos_global` reconstruyó los pedidos distribuidos usando `UNION ALL`, mientras que `clientes_global` reconstruyó los clientes mediante `JOIN` entre datos públicos y datos de contacto. La creación y prueba inicial de vistas se observa en la Evidencia 17.

```
PS D:\Programas\Practica_fragmentacion_07-07\practica-fragmentacion> Get-Content .\sql\06_vistas_globales.sql | docker exec -i pg-campus psql -U admin -d cafeteria
CREATE EXTENSION
CREATE SERVER
CREATE SERVER
CREATE USER MAPPING
CREATE USER MAPPING
CREATE FOREIGN TABLE
CREATE FOREIGN TABLE
CREATE VIEW
 sede | total
-----+-----
 Campus | 1.00
 Ventanas | 1.25
(2 rows)
```

Evidencia 17: Creación de objetos remotos y vista global de pedidos.

La Evidencia 18 corresponde a la ejecución del script de verificación, donde se revisan las tres condiciones principales de la fragmentación.

```

PS D:\Programas\Practica_fragmentacion_07-07\practica-fragmentacion> Get-Content .\sql\07_verificacion.sql | docker exec -i pg-campus psql -U admin -d cafeteria
filas_globales
-----
      8
(1 row)

sede | sum
-----+-----
Babahoyo | 4.25
Campus | 4.25
Ventanas | 2.25
(3 rows)

pedido_id | veces
-----+-----
(0 rows)

```

Evidencia 18: Ejecución del script de verificación.

La existencia de las vistas se verificó con \dv. En la Evidencia 19 se observa que existen `clientes_global` y `pedidos_global`. Luego, la Evidencia 20 muestra que `pedidos_global` reúne los 8 pedidos de los tres nodos, y la Evidencia 21 muestra que `clientes_global` reconstruye los 6 clientes completos.

```

cafeteria=# \dv
              List of relations
 Schema |      Name      | Type | Owner
-----+-----+-----+-----
 public | clientes_global | view | admin
 public | pedidos_global  | view | admin
(2 rows)

cafeteria=#

```

Evidencia 19: Listado de vistas globales creadas en el nodo Campus.

```

cafeteria=# SELECT * FROM pedidos_global;
 pedido_id | cliente_id | producto_id | fecha      | monto | sede
-----+-----+-----+-----+-----+-----
      1 |      1 |      1 | 2026-03-01 |  0.75 | Campus
      3 |      3 |      2 | 2026-03-02 |  1.00 | Campus
      7 |      1 |      3 | 2026-03-04 |  2.50 | Campus
      2 |      2 |      3 | 2026-03-01 |  2.50 | Babahoyo
      6 |      6 |      1 | 2026-03-03 |  0.75 | Babahoyo
      8 |      2 |      2 | 2026-03-04 |  1.00 | Babahoyo
      4 |      4 |      4 | 2026-03-02 |  1.25 | Ventanas
      5 |      5 |      5 | 2026-03-03 |  1.00 | Ventanas
(8 rows)

```

Evidencia 20: Consulta de la vista `pedidos_global`.

```
cafeteria=# SELECT * FROM clientes_global;
 cliente_id | nombre | ciudad | email | telefono
-----+-----+-----+-----+-----
          1 | Maria Alvarado | Quevedo | maria@uteq.edu.ec | 0991111111
          2 | Luis Cedeno | Babahoyo | luis@uteq.edu.ec | 0992222222
          3 | Ana Vera | Quevedo | ana@uteq.edu.ec | 0993333333
          4 | Jose Mendoza | Ventanas | jose@uteq.edu.ec | 0994444444
          5 | Carla Zambrano | Ventanas | carla@uteq.edu.ec | 0995555555
          6 | Pedro Suarez | Babahoyo | pedro@uteq.edu.ec | 0996666666
(6 rows)
```

Evidencia 21: Consulta de la vista `clientes_global`.

Como comparación, las Evidencias 22 y 23 muestran consultas directas sobre las tablas locales `pedidos` y `clientes`. Estas consultas ayudan a comprobar que las vistas globales devuelven la información esperada en comparación con el esquema centralizado.

```
cafeteria=# SELECT * FROM pedidos;
 pedido_id | cliente_id | producto_id | fecha | monto | sede
-----+-----+-----+-----+-----+-----
          1 |          1 |          1 | 2026-03-01 | 0.75 | Campus
          3 |          3 |          2 | 2026-03-02 | 1.00 | Campus
          7 |          1 |          3 | 2026-03-04 | 2.50 | Campus
(3 rows)
```

Evidencia 22: Consulta local de pedidos en el nodo Campus.

```
cafeteria=# SELECT * FROM clientes;
 cliente_id | nombre | ciudad | email | telefono
-----+-----+-----+-----+-----
          1 | Maria Alvarado | Quevedo | maria@uteq.edu.ec | 0991111111
          2 | Luis Cedeno | Babahoyo | luis@uteq.edu.ec | 0992222222
          3 | Ana Vera | Quevedo | ana@uteq.edu.ec | 0993333333
          4 | Jose Mendoza | Ventanas | jose@uteq.edu.ec | 0994444444
          5 | Carla Zambrano | Ventanas | carla@uteq.edu.ec | 0995555555
          6 | Pedro Suarez | Babahoyo | pedro@uteq.edu.ec | 0996666666
(6 rows)
```

Evidencia 23: Consulta de la tabla centralizada de clientes.

6. Verificación de completitud, reconstrucción y disjunción

La verificación final se basó en tres consultas. La primera consulta contó las filas de `pedidos_global` y devolvió 8 registros, lo que confirma que no se perdieron pedidos. La

segunda consulta agrupó los montos por sede y permitió comprobar que la reconstrucción devuelve totales coherentes. La tercera consulta buscó `pedido_id` repetidos y devolvió cero filas, lo que confirma la disjunción. Estos resultados se observan en la Evidencia 24.

```
cafeteria=# SELECT COUNT(*) AS filas_globales
cafeteria=# FROM pedidos_global;
 filas_globales
-----
              8
(1 row)

cafeteria=# SELECT sede, SUM(monto)
cafeteria=# FROM pedidos_global
cafeteria=# GROUP BY sede;
   sede   | sum
-----+-----
Babahoyo | 4.25
Campus   | 4.25
Ventanas | 2.25
(3 rows)

cafeteria=# SELECT pedido_id,
cafeteria=# COUNT(*) AS veces
cafeteria=# FROM pedidos_global
cafeteria=# GROUP BY pedido_id
cafeteria=# HAVING COUNT(*) > 1;
 pedido_id | veces
-----+-----
(0 rows)
```

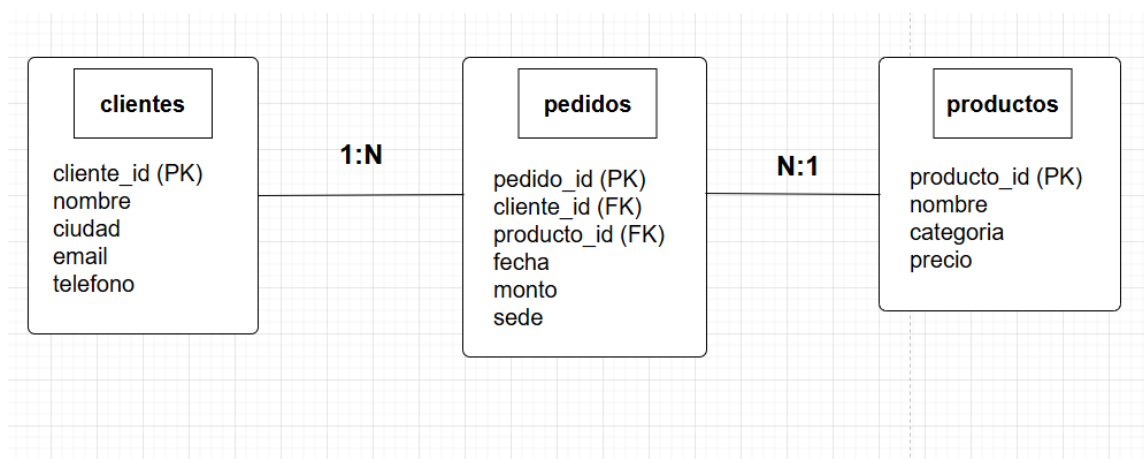
Evidencia 24: Verificación final: conteo global, sumatoria por sede y búsqueda de duplicados.

Cuadro 1: Resumen de las condiciones verificadas.

Condición	Consulta realizada	Resultado obtenido
Compleitud	Conteo de filas en <code>pedidos_global</code>	8 filas
Reconstrucción	Suma de monto agrupada por sede	Babahoyo 4.25, Campus 4.25, Ventanas 2.25
Disjunción	Búsqueda de <code>pedido_id</code> repetidos	0 filas

7. Modelo entidad-relación utilizado

La Evidencia 25 muestra el modelo entidad-relación utilizado en la práctica. La tabla **clientes** almacena la información de los estudiantes registrados, la tabla **productos** contiene los productos disponibles en la cafetería y la tabla **pedidos** registra cada venta realizada. Un cliente puede realizar muchos pedidos, mientras que cada pedido pertenece a un solo cliente. De la misma forma, un producto puede aparecer en muchos pedidos, pero cada pedido registra un solo producto.



Evidencia 25: Modelo entidad-relación usado para la práctica.

A partir de este modelo se aplicó fragmentación horizontal sobre **pedidos** usando la columna **sede**. También se aplicó fragmentación vertical sobre **clientes**, separando los datos públicos de los datos de contacto. Finalmente, la fragmentación mixta combinó ambos criterios sobre **clientes**, dividiendo primero por ciudad y después por sensibilidad de los datos.

8. Preguntas de análisis

8.1. Nueva sede en El Empalme

Si se abre una nueva sede en El Empalme, se tendría que crear un nuevo nodo o contenedor para esa sede. También sería necesario crear el fragmento local de pedidos, configurar el servidor remoto con **postgres_fdw** desde **pg-campus** y actualizar la vista **pedidos_global** para incluir el nuevo fragmento mediante otro **UNION ALL**. El esfuerzo sería moderado, porque no cambia el modelo lógico principal, pero sí se requiere modificar la configuración de infraestructura y las vistas globales.

8.2. Seguridad y fragmentación vertical

La fragmentación vertical mejora la seguridad cuando se separan datos públicos de datos sensibles. Por ejemplo, en una base de datos de empleados se podría guardar **id**, **nombre** y **puesto** en una tabla accesible para consultas generales, mientras que **sueldo**, **número de cuenta bancaria** o **evaluaciones internas** quedarían en otro fragmento con permisos más estrictos. De esta manera, si un usuario accede solo al fragmento público, no podrá consultar información sensible.

8.3. Importancia de replicar la clave primaria

Replicar la clave primaria en la fragmentación vertical permite reconstruir la fila original mediante un `JOIN`. Si la clave primaria no se repitiera en los fragmentos, no existiría una forma segura de saber qué datos de contacto pertenecen a cada cliente. Esto afectaría la integridad de los datos y haría imposible reconstruir la relación original sin ambigüedad.

8.4. Fragmentación horizontal por sede

La fragmentación horizontal por sede es razonable porque los pedidos están naturalmente relacionados con el lugar donde se realizaron. En un escenario real, es común que cada sede consulte más sus propios pedidos que los pedidos de otras sedes. Fragmentar por `producto_id` no sería tan conveniente, porque los productos pueden venderse en cualquier sede y las consultas por ventas locales tendrían que revisar varios nodos.

8.5. Tolerancia a fallos

Para soportar tolerancia a fallos se debería agregar replicación. Cada nodo principal podría tener una réplica que permita seguir consultando la información si el nodo original se cae. Esto aumentaría la disponibilidad del sistema y reduciría el riesgo de pérdida temporal de acceso a los datos.

9. Reflexión final

La práctica permitió entender que fragmentar una base de datos no consiste solamente en dividir tablas, sino en mantener la posibilidad de reconstruir la información sin pérdida ni duplicación. La parte más sencilla fue la fragmentación horizontal, porque la columna `sede` permitió decidir con claridad qué pedidos pertenecían a cada nodo. La fragmentación vertical exigió más cuidado, ya que fue necesario mantener la clave primaria en ambos fragmentos para poder unirlos después.

Una dificultad importante apareció en la fragmentación mixta. Aunque la guía pedía un solo archivo `05_fragmentacion_mixta.sql`, el contenido debía aplicarse en nodos diferentes. Por eso fue más seguro crear tres archivos separados para ejecutar cada parte en su respectivo contenedor. Esta decisión evitó que un nodo recibiera tablas que no le correspondían y permitió documentar mejor la distribución.

Otra dificultad estuvo en la ejecución de scripts desde PowerShell. El comando con redirección `<` generó error, por lo que se usó `Get-Content` para enviar el contenido de los archivos SQL hacia `docker exec`. Esta solución permitió continuar con la práctica sin cambiar de terminal.

Finalmente, las vistas globales mostraron por qué la reconstrucción es una condición esencial. Aunque los datos estaban distribuidos en diferentes contenedores, `postgres_fdw` permitió consultar la información desde `pg-campus` como si estuviera reunida en un solo lugar. Con la verificación final se comprobó que la fragmentación cumplía las condiciones de completitud, reconstrucción y disjunción.

10. Conclusiones

Se implementó correctamente una base de datos distribuida usando tres contenedores PostgreSQL. El esquema centralizado sirvió como referencia, la fragmentación horizontal separó los pedidos por sede, la fragmentación vertical separó datos públicos y de contacto, y la fragmentación mixta combinó ambos criterios. Además, las vistas globales permitieron reconstruir la información y las consultas de verificación confirmaron que los datos no se perdieron ni se duplicaron.

Referencias

- [1] M. T. Özsu y P. Valduriez, *Principles of Distributed Database Systems*. Springer, 2020.
- [2] S. Ceri, S. B. Navathe y G. Wiederhold, “Distribution design of logical database schemas,” *IEEE Transactions on Software Engineering*, vol. SE-9, no. 4, pp. 487–504, 1983.
- [3] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, 2024.